MongoDB(Advanced)

Objective

This is the next phase of MongoDB learning. Trainees should first complete or review the MongoDB Basic training material before starting this module. By the end of this training, trainees will be able to choose practical MongoDB designs, update nested data safely, build useful aggregation pipelines, improve slow queries with indexes, use transactions when needed, and explain how MongoDB supports reliability and scale in production.

Outline

1. Advanced Schema Design
    a. Design Around Access Patterns
    b. Embedding or Referencing?
    c. Avoid Unbounded Arrays
    d. Schema Validation
    e. Useful Design Patterns
2. Advanced Querying and Write Operations
    a. Match One Array Element with `$elemMatch`
    b. Update a Matching Array Element
    c. Group Writes with `bulkWrite()`
    d. Use Range-Based Pagination for Large Lists
3. Advanced Aggregation
    a. Turn Array Values into Rows with `$unwind`\
    b. Join Departments with `$lookup`
    c. Return Data and a Count with `$facet`
    d. Keep Pipelines Efficient
4. Advanced Indexing and Query Performance
    a. Choose Compound Index Order with ESR
    b. Indexes for Specific Needs
    c. Read `explain("executionStats")`
    d. Remember the Write Cost
5. Transactions and Atomicity
    a. Keep Transactions Small
6. Read and Write Consistency
    a. Write Concern
    b. Read Concern
    c. Read Preference
7. Replication and High Availability
    a. What Happens When the Primary Fails?
    b. Replication Lag and Secondary Reads
8. Sharding and Horizontal Scaling
    a. Main Parts of a Sharded Cluster
    b. How Data Is Distributed
    c. Choose the Shard Key Carefully

    d. Scatter-Gather and the Balancer
9. Change Streams
    a. Watch a Collection
    b. Common Uses
10. Backup and Restore
    a. Create a Logical Backup
    b. Restore into a Safe Target
    c. Verify the Backup
11. Monitoring and Performance Troubleshooting
    a. A Simple Slow-Query Workflow
    b. Profiler and Active Operations
    c. What Developers Should Watch
12. Advanced Security and Production Best Practices
    a. Use the Smallest Useful Role
    b. Protect Every Connection
    c. Production Checklist

Introduction

MongoDB Advanced is about making better developer decisions. Instead of learning every option, you will practise a small set of techniques that solve common application problems.

Most examples use a company database and an employees collection so that the data stays familiar throughout the module.

```
use company
db.employees.insertMany([
  { _id: 1, name: "John", departmentId: 10, skills: ["Java"], experience: 4,
active: true },
  { _id: 2, name: "Alice", departmentId: 10, skills: ["Python", "MongoDB"],
experience: 6, active: true },
  { _id: 3, name: "David", departmentId: 20, skills: ["Communication"], experience:
3, active: false }
])
```

> 💡 **Key Takeaway:** Focus on the reason behind each feature: what problem it solves, when to use it, and what trade-off it introduces.

1. Advanced Schema Design

A good MongoDB schema follows the way the application reads and updates data. Start with the most common screens, API calls, and reports not with a relational table diagram.

Design Around Access Patterns

Imagine an employee profile page that always shows the employee and an emergency contact. Keeping the contact inside the employee document makes that read simple.

```
{
  _id: 1,
  name: "John",
  emergencyContact: { name: "Meera", phone: "9999999999" }
}
```

A department, however, is shared by many employees and changes independently. Store departmentId in each employee and keep departments in a separate collection.

Embedding or Referencing?

- Embed when the child data belongs to one parent, is small, and is usually read with the parent.
- Reference when the data is shared, changes independently, or can grow without a clear limit.
- Duplicate a small value only when it makes an important read simpler, and plan how that copy will stay current.

Avoid Unbounded Arrays

An array containing every login ever made by an employee will keep growing. MongoDB limits each BSON document to 16 MiB, so an unbounded array may eventually reach this limit. Large arrays also make documents heavier to read and update. Store unlimited history in a separate collection, or keep only a small recent subset in the employee document.

Schema Validation

MongoDB is flexible, but important fields should still have predictable types. The following validator requires a name and an integer departmentId.

```
db.createCollection("employees", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "departmentId"],
      properties: {
        name: { bsonType: "string" },
        departmentId: { bsonType: "int" }
      }
    }
  }
})
```

Useful Design Patterns

- Extended Reference: copy a small value such as departmentName when it is displayed very often.
- Subset: keep the latest three notes in the employee document and older notes elsewhere.
- Bucket: group many small events, such as one day of attendance entries, into one document.
- Computed: save a value such as monthlyAttendanceCount when recalculating it on every read is expensive.

> 💡 **Key Takeaway:** A flexible schema is not an unplanned schema. Keep together what is used together, and separate data that is shared or grows without a limit.

2. Advanced Querying and Write Operations

Match One Array Element with $elemMatch

Use $elemMatch when several conditions must be true for the same object inside an array.

```
db.employees.find({
  certifications: {
    $elemMatch: { name: "MongoDB", status: "Active" }
  }
})
```

Update a Matching Array Element

The positional $ operator updates the first array element matched by the query.

```
db.employees.updateOne(
  { _id: 1, "certifications.name": "MongoDB" },
  { $set: { "certifications.$.status": "Active" } }
)
```

Array filters are useful when more than one matching element may need an update.

```
db.employees.updateOne(
  { _id: 1 },
  { $set: { "certifications.$[c].status": "Renewal Due" } },
  { arrayFilters: [{ "c.expiryYear": { $lt: 2027 } }] }
)
```

Group Writes with bulkWrite()

bulkWrite() sends several write operations together. It is useful for imports, cleanup jobs, and scheduled updates.

```
db.employees.bulkWrite([
  { updateOne: {
      filter: { name: "John" },
      update: { $inc: { experience: 1 } }
  }},
  { updateOne: {
      filter: { name: "Alice" },
      update: { $set: { active: false } }
  }},
  { insertOne: {
      document: { _id: 4, name: "Emma", departmentId: 20, active: true }
  }}
])
```

Use Range-Based Pagination for Large Lists

skip() and limit() are easy for small lists, but a large skip makes MongoDB walk past many records. For a long employee list, return the last _id to the client and continue after it.

```
db.employees
  .find({ _id: { $gt: 20 } })
  .sort({ _id: 1 })
  .limit(10)
```

> 💡 **Key Takeaway:** Use $elemMatch for one array object, array filters for controlled updates, bulkWrite() for batches, and range-based pagination for large result sets.

3. Advanced Aggregation

MongoDB Basic introduced $match, $group, $project, $sort, and $limit. This section adds stages that reshape arrays, join collections, calculate fields, and return more than one result set.

Turn Array Values into Rows with $unwind

The following pipeline counts how many employees have each skill.

```
db.employees.aggregate([
  { $unwind: "$skills" },
  { $group: { _id: "$skills", employeeCount: { $sum: 1 } } },
  { $sort: { employeeCount: -1 } }
])
```

Join Departments with $lookup

```
db.departments.insertMany([
  { _id: 10, name: "Engineering" },
  { _id: 20, name: "HR" }
])
db.employees.aggregate([
  { $lookup: {
      from: "departments",
      localField: "departmentId",
      foreignField: "_id",
      as: "department"
  }},
  { $unwind: "$department" },
  { $set: { departmentName: "$department.name" } },
  { $unset: "department" }
])
```

Return Data and a Count with $facet

$facet runs small pipelines on the same input. This example returns five active employees and their total count.

```
db.employees.aggregate([
  { $match: { active: true } },
  { $facet: {
      results: [{ $sort: { name: 1 } }, { $limit: 5 }],
      summary: [{ $count: "activeEmployees" }]
  }}
])
```



Keep Pipelines Efficient

- Filter early with $match so later stages process fewer documents.
- Remove fields that are not needed, especially before expensive stages.
- Place an index behind important filters and sorts when the pipeline can use it.
- Test the full pipeline with realistic data, not only a few sample documents.

Memory and Output Limits

Some stages, such as `$group` and a `$sort` that cannot use an index, may require more than 100 MB of memory. Depending on the server configuration, MongoDB may either use temporary disk space or return

a memory-limit error. For supported stages, `{ allowDiskUse: true }` allows the pipeline to use temporary disk space. However, disk processing can be slower, so optimize the pipeline first.

Each document returned by an aggregation pipeline must not exceed the 16 MiB BSON document limit. Avoid creating one large result document containing too many fields or array items. Use stages such as `$project`, `$unset`, or `$unwind` to reduce the document size when required.

> 💡 **Key Takeaway:** Build a pipeline as small steps. Filter first, reshape only what you need, and check performance with realistic data.

4. Advanced Indexing and Query Performance

Choose Compound Index Order with ESR

For many compound indexes, a useful starting point is Equality, Sort, Range (ESR). Put exact-match fields first, then sort fields, then range fields.

```
db.employees
  .find({ departmentId: 10, experience: { $gte: 3 } })
  .sort({ name: 1 })

db.employees.createIndex({
  departmentId: 1,
  name: 1,
  experience: 1
})
```

Here departmentId is Equality, name is Sort, and experience is Range. ESR is a guideline; verify the result with explain().

Indexes for Specific Needs

- Multikey index: created automatically when an indexed field contains an array, such as skills.
- Partial index: includes only documents that meet a filter, such as active employees.
- TTL index: removes documents after a date, which is useful for sessions or temporary data.
- Covered query: MongoDB can answer from the index when every requested field is in that index.

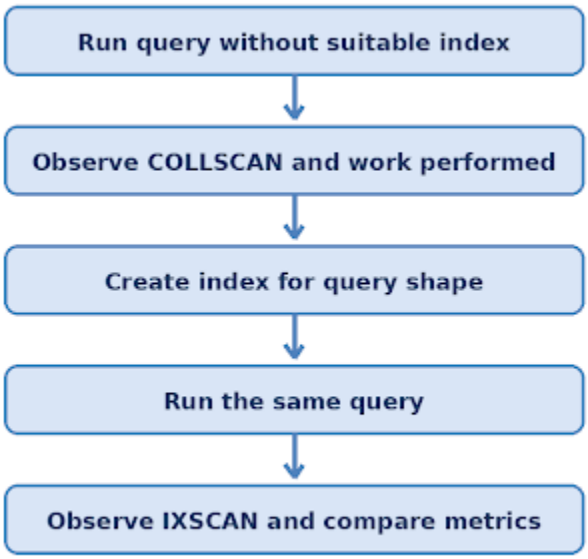
```
db.employees.createIndex(
  { departmentId: 1, name: 1 },
  { partialFilterExpression: { active: true } }
)

db.sessions.createIndex(
  { expiresAt: 1 },
  { expireAfterSeconds: 0 }
)
```

Read explain("executionStats")

```
db.employees
  .find({ departmentId: 10 })
  .explain("executionStats")
```

- COLLSCAN means MongoDB scanned collection documents.
- IXSCAN means MongoDB used an index.
- nReturned is the number of results sent back.
- totalDocsExamined shows how many documents were checked.
- totalKeysExamined shows how many index entries were checked.



A good result usually examines close to the number of documents returned. Do not add an index only because you see COLLSCAN; first confirm that the query is important and frequent.

Remember the Write Cost

Every insert and update must maintain each relevant index. Too many indexes use storage, slow writes, and make index choices harder. Keep indexes that support real access patterns.

> 💡 **Key Takeaway:** Design indexes for real queries, use ESR as a starting point, and use explain() to prove that the index improves useful work.

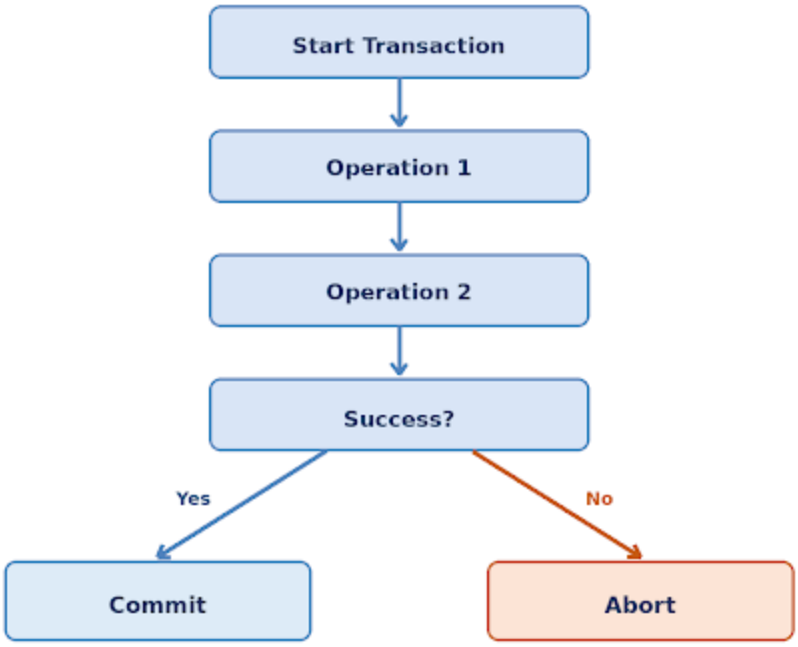
5. Transactions and Atomicity

MongoDB updates one document atomically. If all related changes fit safely in one document, a transaction is usually unnecessary.

Use a multi-document transaction when several documents must either all change or none of them should change. A money transfer is a clear example.

```
const session = db.getMongo().startSession()
const accounts = session.getDatabase("company").accounts

session.startTransaction()
try {
  accounts.updateOne({ name: "Alice" }, { $inc: { balance: -500 } })
  accounts.updateOne({ name: "Bob" }, { $inc: { balance: 500 } })
  session.commitTransaction()
} catch (error) {
  session.abortTransaction()
} finally {
  session.endSession()
}
```



Keep Transactions Small

- Transactions require a replica set or sharded cluster.
- Keep the number of operations and transaction time low.
- Drivers may need to retry some transaction errors.
- Do not use a transaction to replace good document design.
- Avoid a transaction when one atomic document update can solve the problem.

> 💡 **Key Takeaway:** Use transactions for a real all-or-nothing rule across documents, not as the default for every write.

6. Read and Write Consistency

Write Concern

Write concern controls what MongoDB must confirm before the application treats a write as successful.

- w: 1, waits for acknowledgement from the primary.
- w: "majority", waits until most voting data-bearing members have acknowledged the write.
- j: true, asks MongoDB to acknowledge after the write reaches the journal.

```
db.employees.insertOne(
  { _id: 5, name: "Sara", departmentId: 20 },
  { writeConcern: { w: "majority", j: true } }
)
```

Read Concern

- local returns the node's current data and is the usual default.
- majority returns data acknowledged by a majority of replica-set members.
- snapshot gives a consistent view for supported transaction and snapshot-read scenarios.

Read Preference

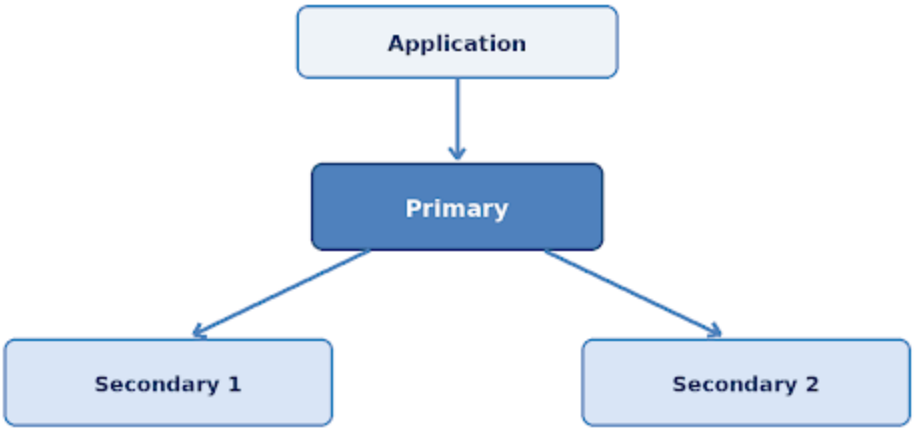
- primary sends reads to the primary and is the safest default when fresh data matters.
- secondary can reduce primary read load but may return slightly older data.
- primaryPreferred and secondaryPreferred provide fallback behavior.

Choose consistency based on the user action. A balance screen needs fresher data than an analytics dashboard that can accept a small delay.

> 💡 **Key Takeaway:** Stronger acknowledgement and fresher reads usually cost more time. Choose the level that matches the business risk.

7. Replication and High Availability

A replica set is a group of MongoDB servers that keep copies of the same data. One member is Primary and receives writes. Secondary members copy the Primary's operations.



What Happens When the Primary Fails?

- The remaining members detect that the Primary is unavailable.
- Eligible members hold an election.
- One Secondary becomes the new Primary.
- The application reconnects through the replica-set connection information.

Failover is automatic, but it is not instant. Applications should expect a short interruption and use drivers that can discover the new Primary.

Replication Lag and Secondary Reads

A Secondary can be slightly behind the Primary. This delay is replication lag. Reading from a Secondary can help read-heavy workloads, but the application must accept that a recent update may not appear immediately.

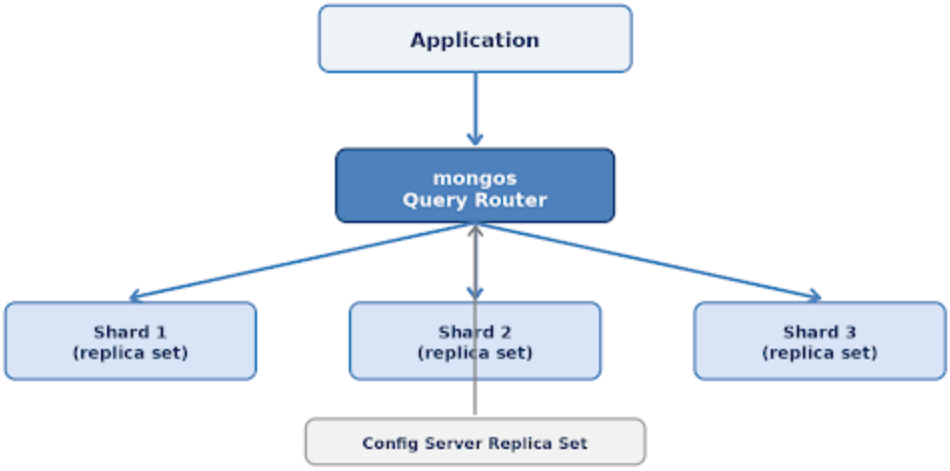
```
db.getMongo().setReadPref("secondaryPreferred")
```

Use secondary reads only for suitable work, such as reports or dashboards. Keep operations that require the newest value on the Primary.

> 💡 **Key Takeaway:** Replica sets improve availability and keep redundant copies, but applications still need retry-friendly connections and a plan for slightly stale secondary reads.

8. Sharding and Horizontal Scaling

Vertical scaling gives one server more CPU, memory, or storage. Horizontal scaling spreads data and works across several servers. MongoDB uses sharding for horizontal scaling.



Main Parts of a Sharded Cluster

Think of a sharded cluster like a library system with multiple branches.

- Shards — each branch holds a portion of the books (data). Each shard is normally a replica set.
- Config servers — the central catalog that knows which branch has which books (cluster metadata).
- mongos — the librarian at the front desk who checks the catalog and sends you to the right branch. It receives application requests and routes them to the correct shard.

How Data Is Distributed

MongoDB splits each collection's data into chunks. A chunk is a contiguous range of shard key values. MongoDB distributes these chunks across shards.

There are two sharding strategies:

- Range-based sharding: chunks are divided by value ranges. Good for range queries, but a steadily increasing key (such as a timestamp) can send all new writes to the last chunk on one shard.
- Hash-based sharding: MongoDB hashes the shard key value before assigning chunks. This gives a more even distribution, but range queries become slower because nearby values land on different shards.

Choose the Shard Key Carefully

The shard key is the field MongoDB uses to decide which shard receives each document. A useful shard key has many possible values, spreads writes and storage, and appears in important queries.

✅ Good direction: a high-cardinality key (many unique values) that matches common query filters and distributes traffic evenly. Example: customer_id.

❌ Poor choice: department, because only a few department values may exist. Most data lands on the same few shards.

⚠️ Risky choice: a steadily increasing value like created_at for a range-sharded collection, because all new writes collect on one shard.

Scatter-Gather and the Balancer

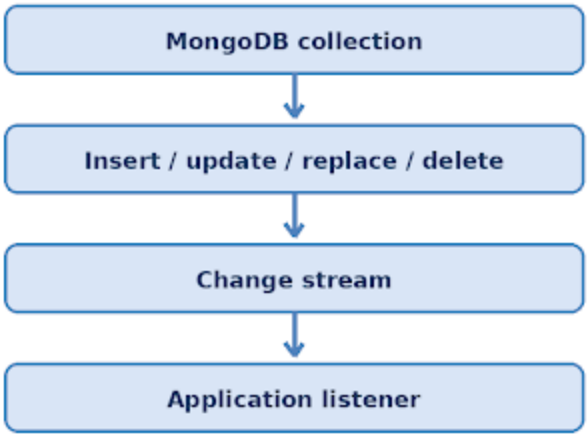
If a query does not include the shard key, mongos cannot route it to one shard. Instead, it asks every shard and merges the results. This is called scatter-gather, and it is slower than a targeted query.

MongoDB's balancer automatically moves chunks between shards to keep storage and traffic even. However, balancing cannot fully repair a shard key that sends most traffic to one value. The shard key choice matters from the start.

> 💡 **Key Takeaway:** Sharding can add capacity, but the shard key decides whether data and traffic are distributed well. Understand ranged versus hashed strategies, and choose a key with high cardinality that matches your common queries.

9. Change Streams

A change stream lets an application listen for inserts, updates, replacements, and deletes without repeatedly polling the collection.
Prerequisite: Change streams require MongoDB to run as a replica set or sharded cluster. They are not available on a standalone MongoDB deployment.



Watch a Collection

```
const stream = db.employees.watch()

while (stream.hasNext()) {
  printjson(stream.next())
}
```

A change event includes operationType, a documentKey, and a resume token in the _id field. Insert and replace events can include the new document; update events describe changed fields unless full-document lookup is requested.

Common Uses

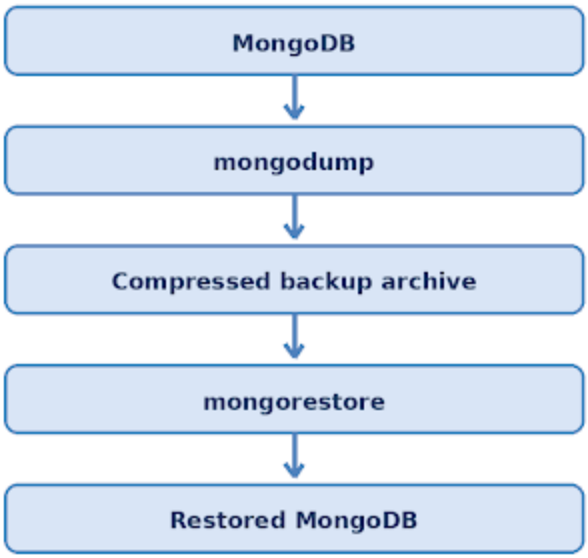
- Send a notification when an important record changes.
- Update a cache after a document changes.
- Write audit or event-processing logic.
- Feed an event-driven application.

Store the resume token when reliable continuation matters. After a restart, the listener can continue from that position instead of starting again.

> 💡 **Key Takeaway:** Change streams turn database changes into application events. Handle restarts and resume tokens before relying on them in production.

10. Backup and Restore

High availability keeps an application running when a server fails. A backup protects against deleted or damaged data. Production systems usually need both.



Create a Logical Backup

```
mongodump --db company --archive=company.archive --gzip
```

Restore into a Safe Target

```
mongorestore --archive=company.archive --gzip \
  --nsFrom="company.*" \
  --nsTo="company_restore.*"
```

Restoring into company_restore avoids overwriting the working database during a practice run.

Verify the Backup

```
use company_restore
db.employees.countDocuments()
db.employees.findOne({ name: "Alice" })
```

- Run backups on a schedule that matches acceptable data loss.
- Protect backup files and limit access to them.
- Test restores regularly; a backup is useful only when it can be restored.
- Record the restore steps so they are available during an incident.

> 💡 **Key Takeaway:** Replication is not a backup. Test a restore into an isolated database and verify important records.

11. Monitoring and Performance Troubleshooting

Monitoring helps developers find problems before users experience long delays. Start with the symptom, collect evidence, make one change, and measure again.

A Simple Slow-Query Workflow

- Capture the slow query and its typical input.
- Run explain("executionStats") with realistic data.
- Check for COLLSCAN, high documents examined, or an expensive sort.
- Create or adjust one suitable index.
- Run the same test again and compare the numbers.

Profiler and Active Operations

```
db.setProfilingLevel(1, { slowms: 100 })

db.system.profile
  .find()
  .sort({ ts: -1 })
  .limit(5)

db.currentOp({ active: true })

db.setProfilingLevel(0)
```

What Developers Should Watch

- Query time and the number of documents examined
- Connections, CPU, memory, and disk usage
- Replication lag
- Large documents and unbounded arrays
- Large skip() values
- Too many indexes or expensive aggregation stages

> 💡 **Key Takeaway:** Troubleshooting is a measure–change–measure loop. Avoid guessing; use explain(), profiler data, and system metrics.

12. Advanced Security and Production Best Practices

Use the Smallest Useful Role

Role-Based Access Control (RBAC) decides what each user can do. Built-in roles such as read and readWrite cover common needs. Use a custom role only when the application needs a narrower set of actions.

Protect Every Connection

- Use TLS so network traffic is encrypted.
- Restrict network access to trusted application hosts and administrators.
- Use encryption at rest where required by the environment.
- Keep credentials in environment configuration or a secrets manager.
- Never use an administrative user as the application's normal account.

```
MONGODB_URI="mongodb+srv://appUser:<password>@cluster.example/company?retryWrites=true&w=majority"
```

Do not commit the real connection string to source control. Rotate credentials when staff, systems, or risk conditions change.

Production Checklist

- Use a replica set and test failover behavior.
- Create indexes from measured query patterns.
- Set backup schedules and test restores.
- Monitor slow queries, resources, and replication lag.
- Set timeouts and retry behavior in the application driver.
- Review roles and network access regularly.

> 💡 **Key Takeaway:** Production security is a set of layers: least privilege, encrypted connections, restricted networks, protected secrets, backups, and monitoring.

Gem

- https://gemini.google.com/u/3/gem/5aed75cf50c1

Reference material

- https://www.mongodb.com/docs/manual/
- https://learn.mongodb.com/pages/mongodb-developer-learning-paths

Advanced reading material (good to have)

- https://www.mongodb.com/docs/manual/data-modeling/
- https://www.mongodb.com/docs/manual/core/aggregation-pipeline-optimization/
- https://www.mongodb.com/docs/manual/core/transactions/
- https://www.mongodb.com/docs/manual/replication/
- https://www.mongodb.com/docs/manual/administration/production-checklist-development/
- https://www.mongodb.com/docs/manual/administration/security-checklist/